Optimised and Sparse Matrix Multiplication in Java

Pablo Castillo Garcerá
Big Data, Individual Assignment 2, 2026-2027
GitHub: https://github.com/Pablo5250/matrix-multiplication (folder assignment2)

1. The problem

In Assignment 1, the basic multiplication with three loops (i-j-k) was very slow for big matrices. Part of the problem was the cache (the small and fast memory inside the CPU): the inner loop reads matrix B by columns, so it jumps around in memory. In this assignment I use Java and I study two questions:

- **Part A:** how much faster is the normal (dense) multiplication if I change the order of the loops or if I work in small blocks?
- **Part B:** when most of the numbers are zero, when is it better to save only the numbers that are not zero (a sparse format)?

Words I use: a dense matrix saves all its numbers. A sparse matrix has almost all its numbers equal to zero. The density is the percentage of numbers that are not zero, and the sparsity is the percentage of zeros (for example, 0.1% density = 99.9% sparsity).

What I expected

- **Changing the loop order to i-k-j will be much faster.** This means that the inner loop reads B and C row by row, in order, so the CPU can use the cache well.
- **Blocking will be the fastest for big matrices.** This means doing the work in small pieces (tiles) that fit in the cache, so the same numbers are used many times before they leave the cache.
- **The sparse format will be much faster when there are very few nonzeros,** because it skips the zeros. But when there are many nonzeros it will be slower, because it needs extra work to find where each number is.

The four methods

Table 1. Methods. The three dense methods do the same n^3 operations and give exactly the same result.

Method	What changes	Expected effect
Baseline i-j-k	Nothing: the same Java code as Assignment 1	Reads B by columns: slow for big n
i-k-j	The two inner loops are swapped	Reads B and C row by row, in order
Blocked	The i-k-j work is done in tiles of 64 × 64 numbers	The tiles stay in the cache
CSR	Saves only the nonzeros and multiplies only nonzero pairs	Less work when there are many zeros

CSR (Compressed Sparse Row) saves a sparse matrix with three arrays [5]: values (the nonzero numbers, row after row), col_index (the column of each value) and row_ptr (where each row starts in values). For example, the 4 × 4 matrix with rows (0 5 0 0), (0 0 8 0), (0 0 0 3), (1 0 0 0) is saved as values = [5, 8, 3, 1], col_index = [1, 2, 3, 0] and row_ptr = [0, 1, 2, 3, 4] [4]. To multiply two CSR matrices, for each row i of A and each nonzero a[i][k], I add a[i][k] × (row k of B) to a work row; this is Gustavson's method [6]. All the code is mine; I did not use a library.

How I did the experiment

- **Same inputs:** the matrices are created in Java with fixed seeds (the starting number of the random generator, so they are always the same). Dense matrices have random numbers between 0 and 1. In

sparse matrices each position is nonzero with a probability equal to the density, so the nonzeros can be anywhere. For Part B, the dense methods get the same matrices converted from CSR.

- **Correct results first:** 63 tests (2×2 and 3×3 by hand, identity and zero matrices, sizes that are not a multiple of 64). The two optimised methods give exactly the same numbers as the baseline, and CSR gives exactly the same numbers as the dense method for densities from 0.1% to 100%. In the big runs, CSR and dense also gave the same total sum of C in all 11 cases where both were run. The baseline is the same code that I checked against NumPy in Assignment 1.
- **What I measure:** only the multiplication, including creating the result. Creating the inputs and converting between dense and CSR are not included; the time to convert dense to CSR is measured separately.
- **Repetitions:** every test runs in a new Java process with 3 warm-up runs that I do not count (Java needs them to compile the code) and 5 measured runs. I report the median (the middle value) and the IQR (how much the runs change). All the code uses one core.
- **Limits:** 20 minutes and 8 GB of memory per test, decided before starting. If a test is predicted to go over, it is skipped.
- **Memory:** the maximum RAM used by the Java process (psutil), and also the size of only the data: 8 bytes per number in a dense matrix, and 12 bytes per nonzero in CSR (8 for the value and 4 for the column).

Table 2. Computer and software.

Item	Value
CPU	AMD Ryzen 9 5900X (12 cores); L2 cache 512 KB per core, L3 cache 32 MB [7]
RAM / OS	32 GB / Windows 11
Java	OpenJDK 25.0.1, option -Xmx8g (maximum heap 8 GB)

2. Results: Part A (dense optimisation)

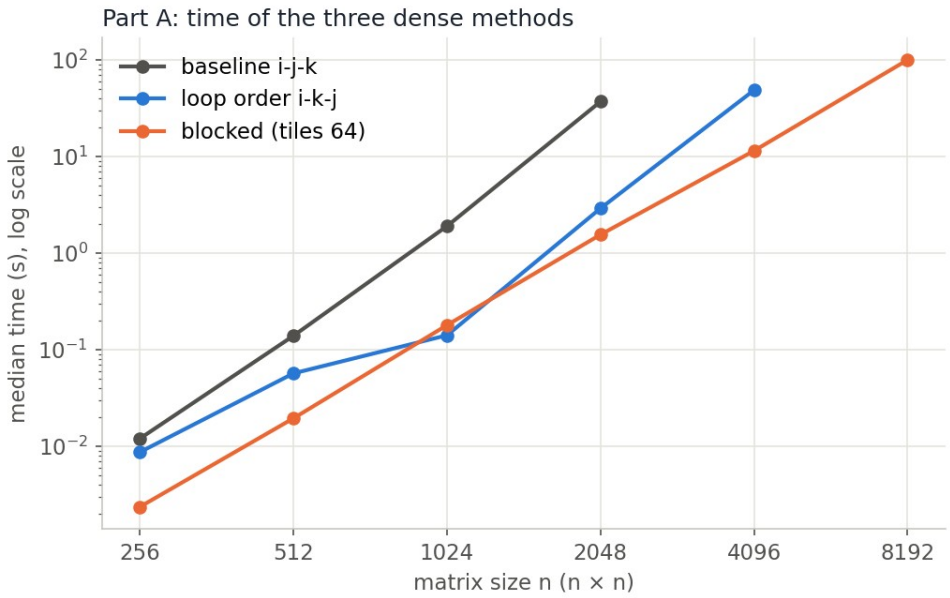


Figure 1. Time of the three dense methods (both axes in logarithmic scale).

In this figure we can see that the baseline is always the slowest: at n = 2048 it needs 37 seconds, and n = 4096 was skipped because it would need about 40 minutes. The i-k-j version is very fast up to n = 1024 (0.14 s), but at 2048 and 4096 it becomes much slower (48.8 s at 4096). The blocked version is the only one that keeps growing like n^3 and it reached n = 8192 in 100 seconds.

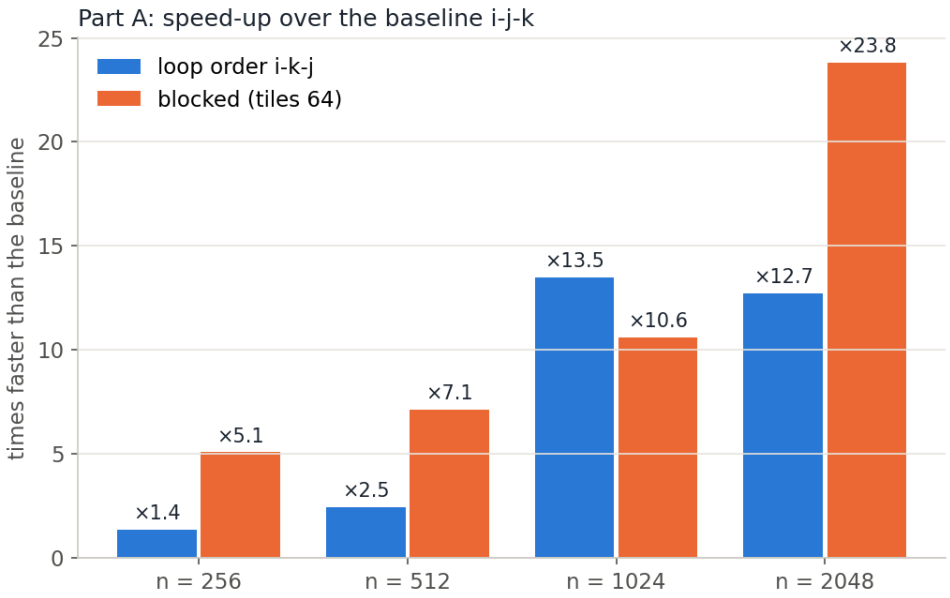


Figure 2. How many times faster each optimised method is than the baseline.

At n = 2048 the blocked method is 23.8 times faster than the baseline, and i-k-j is 12.7 times faster. For small matrices the gain is smaller (i-k-j is only 1.4 times faster at n = 256), because at that size all the data fits in the cache anyway. Why does i-k-j slow down? For every row of C, i-k-j reads the whole matrix B. At n = 2048, B needs 32 MB, the same as the L3 cache, so B has to come again from the slow memory for every row. The blocked method reuses small tiles of B that stay in the cache, so at n = 4096 it is 4.2 times faster than i-k-j.

Table 3. Part A results. "× faster" compares with the baseline at the same n.

Method	n	Runs	Median time	IQR	× faster	Peak memory (MiB)
baseline i-j-k	256	5	12.1 ms	0.120 ms	×1.0	53
baseline i-j-k	512	5	140.2 ms	4.2 ms	×1.0	71
baseline i-j-k	1024	5	1.91 s	32.3 ms	×1.0	118
baseline i-j-k	2048	5	37.17 s	1.37 s	×1.0	302
baseline i-j-k	4096	-	skipped: over the 20 min limit	-	-	-
baseline i-j-k	8192	-	skipped: over the 20 min limit	-	-	-
i-k-j	256	5	8.8 ms	0.204 ms	×1.4	55
i-k-j	512	5	57.2 ms	0.390 ms	×2.5	71
i-k-j	1024	5	141.8 ms	21.4 ms	×13.5	120
i-k-j	2048	5	2.92 s	251.2 ms	×12.7	302
i-k-j	4096	5	48.76 s	4.11 s	-	997
i-k-j	8192	-	skipped: over the 20 min limit	-	-	-
blocked (tile 64)	256	5	2.4 ms	0.027 ms	×5.1	54
blocked (tile 64)	512	5	19.6 ms	0.031 ms	×7.1	71
blocked (tile 64)	1024	5	180.5 ms	20.8 ms	×10.6	120
blocked (tile 64)	2048	5	1.56 s	69.2 ms	×23.8	303
blocked (tile 64)	4096	5	11.50 s	494.4 ms	-	997
blocked (tile 64)	8192	5	100.31 s	2.99 s	-	3,378

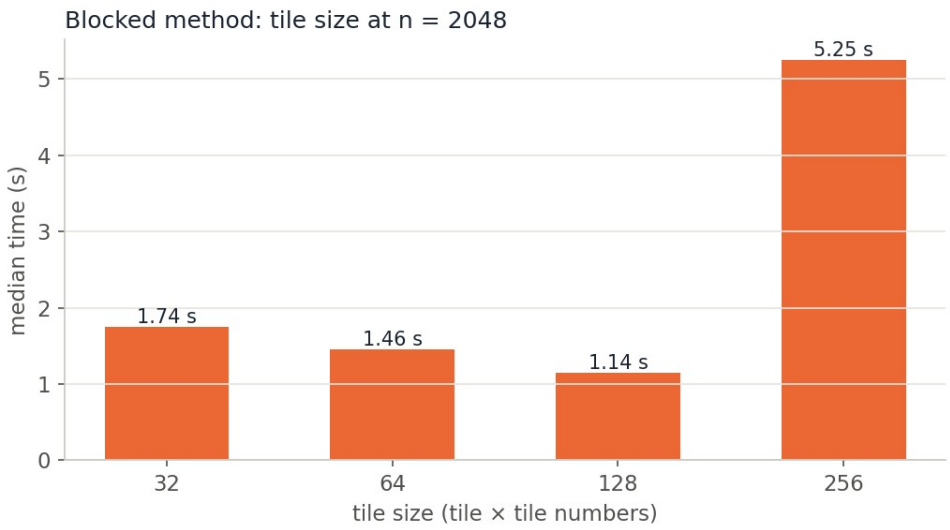


Figure 3. Blocked method with different tile sizes at n = 2048.

I chose tiles of 64 × 64 before measuring, because 64 × 64 numbers are 32 KB, the size of the L1 cache. The test shows that 128 is a bit better here (1.14 s instead of 1.46 s), and 256 is much worse (5.25 s), because three tiles of 256 × 256 numbers need 1.5 MB and do not fit in the L2 cache. So the tile size matters, and the best value depends on the cache of the computer.

3. Results: Part B (sparse matrices)

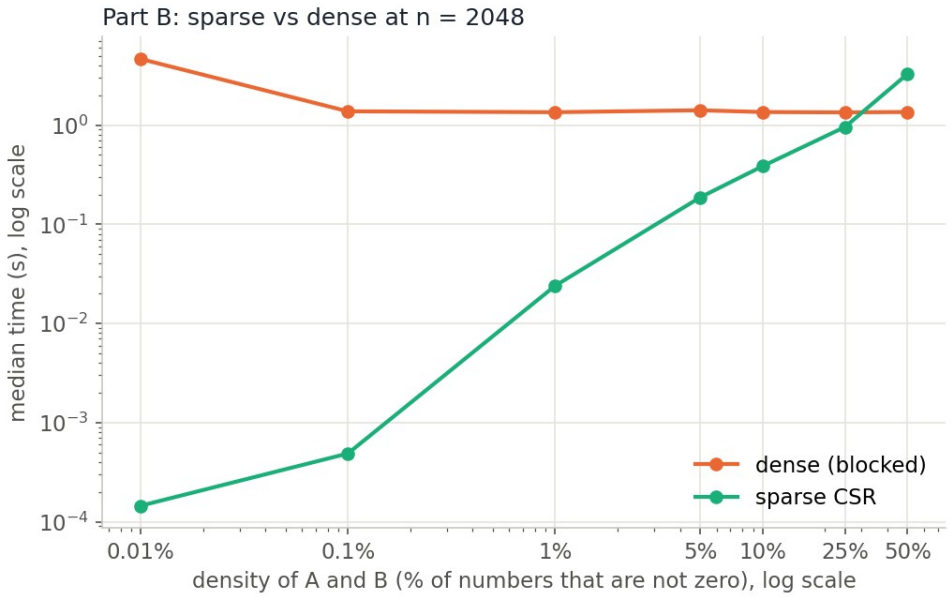


Figure 4. Sparse (CSR) and dense (blocked) at n = 2048 for different densities (both axes in logarithmic scale).

In this figure we can see that the dense time does not change with the density (about 1.35 s), because the dense method multiplies all the numbers, also the zeros. The CSR time grows with the density. CSR is faster up to 25% density (0.96 s against 1.35 s), but at 50% the dense method is faster (1.36 s against 3.28 s). When the matrices are very sparse the difference is huge: at 0.1% CSR needs 0.5 ms, about 2,800 times less than the dense method.

The first dense point (0.01%) is higher (4.6 s) for a reason that is not the density: in that Java process, the JIT compiler did not finish optimising the code during the warm-up runs, so all the runs stayed slow. I kept this

measurement, and I repeated the test twice: both times the time went down to 1.36 s after two runs (results/raw/recheck_dense_n2048_d0.01.csv).

Table 4. Part B at n = 2048. A and B have the same density. "Convert to CSR" = time to convert A from dense to CSR, measured separately.

Density	Nonzeros in A	Nonzeros in C	Density of C	CSR time	Dense time	Convert to CSR
0.01%	409	77	0.00184%	0.145 ms	4.65 s	2.7 ms
0.1%	4,063	8,104	0.193%	0.488 ms	1.38 s	2.5 ms
1%	41,817	776,249	18.5%	24.0 ms	1.35 s	3.0 ms
5%	209,808	4,169,589	99.4%	186.7 ms	1.41 s	5.2 ms
10%	419,323	4,194,304	100%	385.3 ms	1.36 s	7.4 ms
25%	1,047,258	4,194,304	100%	959.0 ms	1.34 s	14.9 ms
50%	2,095,225	4,194,304	100%	3.28 s	1.36 s	27.4 ms

The table shows an important point: **the result is much denser than the inputs.** With 1% nonzeros in A and B, C has 18.5% nonzeros, and with 5% C is almost full (99.4%). This is because each value of C collects many products. So the output costs more and more: with high density, CSR has to save millions of values with their columns and sort the columns of each row. Moreover, converting a dense matrix to CSR takes 2.5 to 27 ms. That is small compared with the dense multiplication, but at 0.1% it is 5 times more than the CSR multiplication itself (2.5 ms against 0.5 ms), so if the data starts dense, the conversion becomes the main cost. Also, CSR does not always save memory: at 50% density the CSR data of A, B and C needs 96 MiB, the same as dense, because every value also needs its column (12 bytes instead of 8).

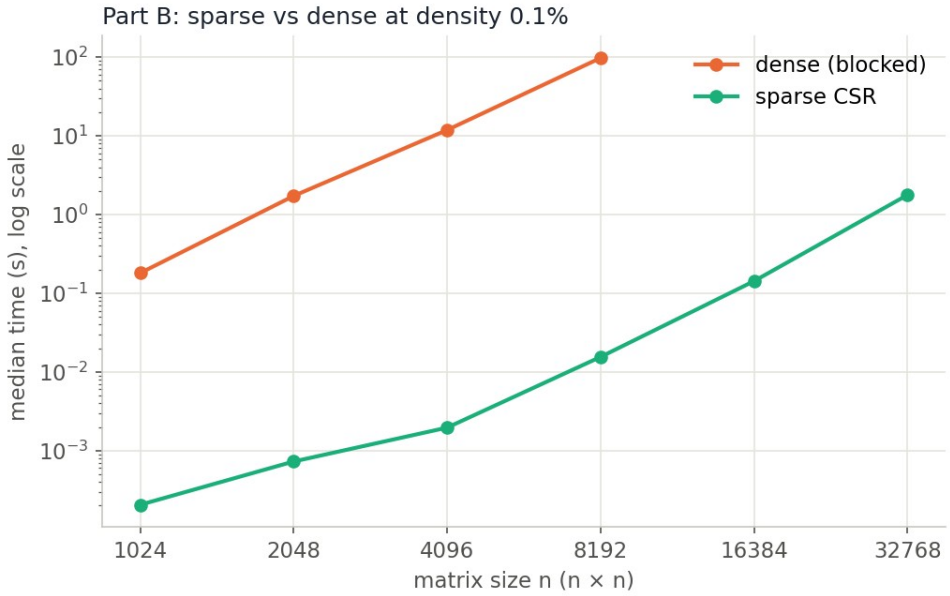


Figure 5. Sparse (CSR) and dense (blocked) at 0.1% density for growing n.

With 0.1% density, CSR is always much faster: at n = 8192 it needs 0.016 s, while the dense method needs 98 s (about 6,000 times more). The dense method stopped at n = 8192, because n = 16384 would need almost 2 hours; CSR reached n = 32768 in 1.8 s. However, the output density also grows with n (from 0.1% to 3.2% at n = 32768), because every row of C collects products from more rows of B.

Table 5. Part B at 0.1% density. Peak = maximum memory of the Java process.

n	Nonzeros in A	Nonzeros in C	Density of C	CSR time	Dense time	CSR peak (MiB)	Dense peak (MiB)
1024	1,065	1,068	0.102%	0.205 ms	180.6 ms	59	118
2048	4,063	8,104	0.193%	0.727 ms	1.72 s	85	303
4096	16,754	67,400	0.402%	2.0 ms	11.86 s	190	1,130
8192	66,831	544,343	0.811%	15.5 ms	97.62 s	235	3,817
16384	267,664	4,352,110	1.62%	142.5 ms	skipped	386	-
32768	1,075,191	34,649,297	3.23%	1.77 s	skipped	8,117	-

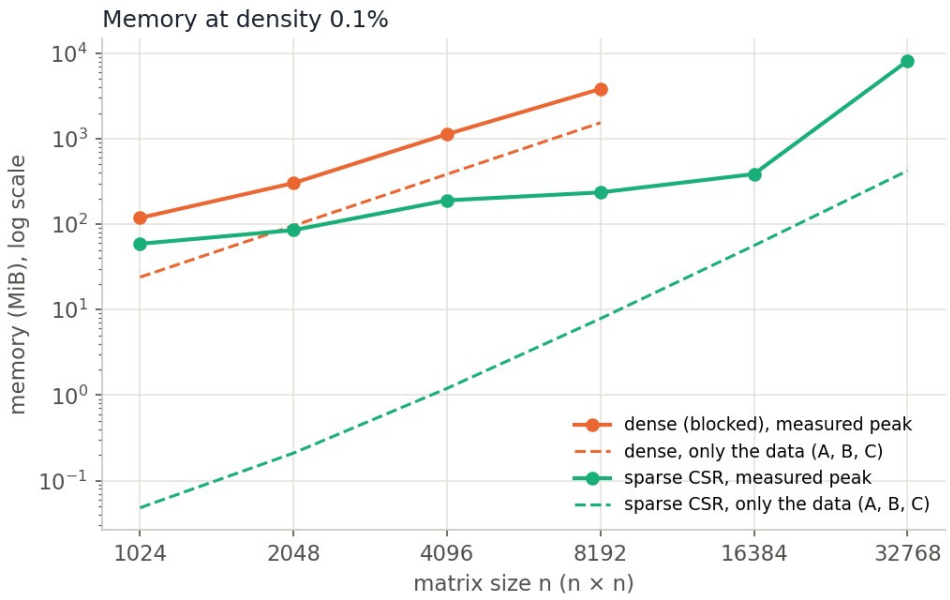


Figure 6. Memory at 0.1% density: measured peak of the process (solid) and size of only the data (dashed).

In this figure we can see that the dense data alone needs 1.5 GB at n = 8192, while the CSR data needs only 8 MB. The measured peaks are much bigger than the data for both methods, because Java keeps the old results until the garbage collector cleans them, and with an 8 GB heap it can wait a long time. That is why CSR at n = 32768 reached 7.9 GB of process memory, close to my 8 GB limit, although its data needs only about 0.4 GB. So in Java the measured memory depends a lot on the JVM settings, not only on the data.

4. Conclusions and future work

On this computer and with these settings:

- **Optimisation (Part A):** a small change in the code has a big effect. Changing only the loop order made the multiplication up to 13.5 times faster, and blocking up to 23.8 times faster, and blocking was the only dense method that reached n = 8192. The number of operations is the same (n^3); the gain comes from reading the memory in a better order.
- **Sparse storage (Part B):** CSR is thousands of times faster and uses much less memory when there are very few nonzeros, but the advantage gets smaller when the density grows, and at 50% the dense method is better.
- **The percentage of zeros alone does not decide:** the size n, how dense the result becomes and the cost of converting the data also matter. Furthermore, the two effects are different: Part A wins by using the cache better, and Part B wins by doing less work.

Limitations: I used only one computer. The nonzeros were placed at random; real sparse matrices often have a structure (for example, numbers near the diagonal), and that can change the results. Java's JIT and garbage collector add noise to the time and to the memory, and I tested the tile size only at one n.

Future work: in Assignment 3 I will split the work between the cores of the CPU (parallel multiplication). It would also be interesting to use tiles of 128 and to test other sparse formats, like CSC, or a sparse matrix multiplied by a dense one.

References

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, 4th ed. MIT Press, 2022.

[2] U. Drepper, “What every programmer should know about memory,” Red Hat, Inc., 2007.

[3] J. L. Hennessy and D. A. Patterson, Computer Architecture: A Quantitative Approach, 6th ed. Morgan Kaufmann, 2017.

[4] Big Data course notes, “Experiments: Monitoring and Performance Engineering,” Grado en Ciencia e Ingeniería de Datos, 2026.

[5] Y. Saad, Iterative Methods for Sparse Linear Systems, 2nd ed. SIAM, 2003.

[6] F. G. Gustavson, “Two fast algorithms for sparse matrices: multiplication and permuted transposition,” ACM Transactions on Mathematical Software, vol. 4, no. 3, pp. 250-269, 1978.

[7] AMD, “Software Optimization Guide for AMD Family 19h Processors,” pub. 56665, 2020.

Code and data: everything is in https://github.com/Pablo5250/matrix-multiplication (folder assignment2): the Java code, the tests, config.json with all the settings, the raw measurements (results/raw/times.csv), and the scripts that make the tables and figures. run_all.bat repeats the whole experiment. The baseline i-j-k is the Java code of Assignment 1 (folder assignment1).

Use of generative AI. I used Claude (Anthropic) as a support tool to help write the code and draft the text of this report. I reviewed all the code and the text, ran every test and measurement on my own computer, and checked the results. I am responsible for the code, the data and the conclusions.